

데이터 전처리

모델에 데이터를 넣기 전에 하는 일들

1. 데이터 전처리가 뭘까	2
2. 빈 값(결측치) 처리	3
3. 튀는 값(이상치) 처리	4
4. 숫자 크기 맞추기와 글자를 숫자로 바꾸기	5
5. 데이터 나누기와 주의할 점	6
6. 이미지 데이터 전처리	7
7. 정리	8

1. 데이터 전처리가 뭘까

요리를 할 때 재료를 바로 냄비에 넣지 않는다. 씻고, 상한 부분은 버리고, 크기에 맞게 썰어 둔다. 데이터도 똑같다. 모은 데이터를 그대로 모델에 넣으면 안 되고, 모델이 먹을 수 있게 손질하는 과정이 필요한데 이것 **데이터 전처리**라고 한다.

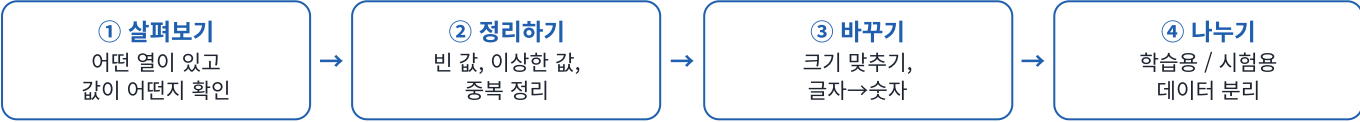
1.1 왜 해야 할까

실제로 모은 데이터는 생각보다 지저분하다.

- 칸이 비어 있다. (설문에서 나이를 안 적은 사람)
- 말도 안 되는 값이 섞여 있다. (나이 999살, 키 -170cm)
- 같은 걸 다르게 적었다. ("서울", "서울시", "Seoul")
- 똑같은 행이 두 번 들어가 있다.

또 모델은 숫자만 계산할 수 있어서 "남/여", "빨강/파랑" 같은 글자나 사진 파일은 숫자로 바꿔 줘야 한다. 이런 걸 정리하지 않고 학습시키면 모델이 이상한 걸 배우게 된다. 그래서 "쓰레기를 넣으면 쓰레기가 나온다(Garbage In, Garbage Out)"는 말이 있다. 실제로 데이터 분석에서 시간이 제일 많이 드는 단계가 모델 만들기보다 이 전처리라고 한다.

1.2 전처리 순서



한 번에 끝나는 건 아니다. 다음 주에 할 탐색적 데이터 분석(EDA)에서 그래프를 그려 보다가 문제를 발견하면 다시 돌아와서 고친다.

1.3 표 데이터와 이미지 데이터

데이터는 크게 두 종류로 나뉜다.

	정형 데이터	비정형 데이터
생김새	엑셀처럼 행과 열로 된 표	사진, 영상, 글, 소리
주로 쓰는 모델	머신러닝 (회귀, 랜덤포레스트 등)	딥러닝 (CNN 등)
주로 하는 전처리	빈 값·이상한 값 처리, 크기 맞추기, 글자를 숫자로	사진 크기 통일, 픽셀 값 조정, 사진 변형해서 늘리기
쓰는 도구	pandas , scikit-learn	torchvision , OpenCV

2~5장은 표 데이터를 중심으로, 6장은 이미지 데이터를 중심으로 정리했다. 다만 크기 맞추기, 시험 데이터 따로 두기 같은 생각은 데이터 종류와 상관없이 똑같이 적용된다.

2. 빈 값(결측치) 처리

값이 비어 있는 칸을 **결측치**라고 한다. 파이썬에서는 보통 `NaN` 으로 보인다. 가끔 빈 칸 대신 -1이나 999 같은 값을 넣어 두는 경우도 있어서 그런 것도 같이 찾아야 한다.

2.1 먼저 왜 비었는지 생각해 보기

빈 칸이 생긴 이유에 따라 처리 방법이 달라진다.

경우	예시
그냥 우연히 빠짐	설문지 한 장이 인쇄 실수로 뒷면이 비었다.
다른 값과 관련 있어서 빠짐	나이가 많은 사람일수록 "SNS 사용 시간" 칸을 비워 두었다.
그 값 자체 때문에 빠짐	소득이 아주 높거나 낮은 사람이 소득을 안 적었다.

첫 번째처럼 우연히 빠진 거라면 그 줄을 지워도 큰 문제가 없다. 하지만 두 번째, 세 번째 경우에 그냥 지워 버리면 특정한 사람들만 데이터에서 사라져서 결과가 한쪽으로 치우친다. 이럴 땐 "이 칸이 비어 있었음"을 표시하는 열을 따로 만들어 두는 것도 방법이다.

2.2 처리 방법

방법	어떻게	특징
줄 지우기	빈 칸이 있는 행을 통째로 지운다.	쉽지만 데이터가 줄어든다.
열 지우기	절반 이상 비어 있는 열은 아예 뺀다.	쓸 수 없는 열을 정리할 때
평균·중앙값으로 채우기	숫자 열은 평균이나 중앙값으로 채운다.	가장 많이 쓴다. 이상치가 있으면 중앙값이 낫다.
가장 많은 값으로 채우기	글자 열은 가장 많이 나온 값이나 "모름"으로 채운다.	범주형 데이터에 사용
비슷한 데이터 보고 채우기	나와 비슷한 사람들의 값을 참고해서 채운다. (KNN 방식)	더 정확하지만 계산이 오래 걸린다.
앞뒤 값으로 채우기	날짜 순서가 있는 데이터는 바로 앞 값이나 앞뒤 중간값으로 채운다.	시간 순서가 있을 때만

```
# 빈 값 확인하기
df.isna().sum()

# 얼마나 빈 칸 개수

# 채우기
df['age'] = df['age'].fillna(df['age'].median())
df['job'] = df['job'].fillna('모름')

# 나이는 중앙값으로
# 직업은 '모름'으로

# 지우기
df = df.dropna()
```

평균이나 중앙값은 **학습용 데이터로만** 계산해야 한다. 시험용 데이터까지 섞어서 계산하면 모델이 시험 문제를 살짝 미리 본 셈이 된다. 자세한 건 5장에서 다시 설명한다.

3. 튀는 값(이상치) 처리

다른 값들과 동떨어진 값을 **이상치**라고 한다. 이상치는 두 종류가 있다.

- **잘못 들어간 값**: 나이 999살, 키 -170cm처럼 입력 실수인 경우. 고치거나 지우면 된다.
- **진짜인데 드문 값**: 재벌의 연봉, 카드 부정 사용처럼 실제로 있는 값. 함부로 지우면 안 된다.

3.1 이상치 찾는 법

(1) IQR 방법 (박스플롯)

데이터를 작은 값부터 줄 세웠을 때 25% 지점을 Q1, 75% 지점을 Q3라고 한다. 그 사이 거리(Q3 - Q1)를 IQR이라고 부르고, 이 범위에서 너무 멀리 떨어진 값을 이상치로 본다.

$$\text{정상 범위} = Q1 - 1.5 \times IQR \sim Q3 + 1.5 \times IQR$$

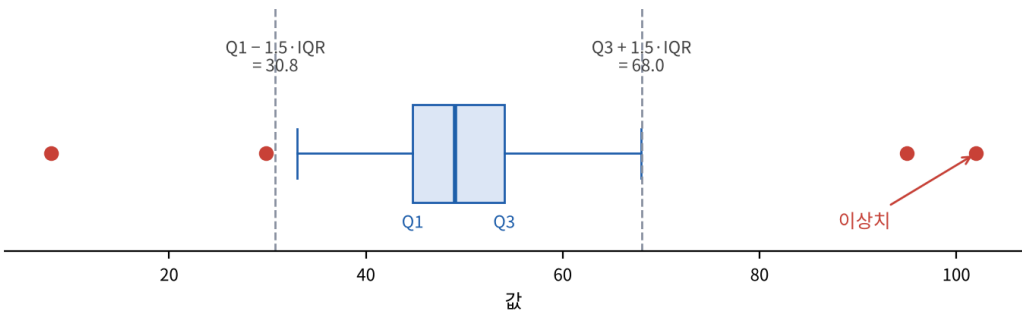


그림 1. 박스플롯. 점선 바깥에 있는 빨간 점들이 이상치다.

(2) Z-점수 방법

값이 평균에서 얼마나 떨어져 있는지를 표준편차 단위로 잰다. 보통 3보다 크면(평균에서 표준편차 3배 이상 떨어지면) 이상치로 본다.

$$z = (\text{값} - \text{평균}) / \text{표준편차}$$

(3) 그래프로 직접 보기

숫자 기준만 믿지 말고 박스플롯이나 히스토그램을 그려서 눈으로 한 번 확인하는 게 좋다. 키 120cm에 몸무게 90kg처럼 하나씩 보면 정상인데 같이 보면 이상한 경우도 있기 때문이다.

3.2 찾은 다음에는

방법	어떻게	언제
지우기	그 행을 삭제한다.	확실한 입력 실수일 때
고치기	원래 값을 알면 고친다. (1700 → 170.0cm)	왜 틀렸는지 알 때
잘라내기	범위를 넘는 값을 경계값으로 바꾼다.	진짜 값이지만 영향을 줄이고 싶을 때
로그 씌우기	큰 값을 작게 눌러 준다. (4장 참고)	연봉, 조회수처럼 몇몇 값만 엄청 클 때
그냥 두기	손대지 않는다.	이상치를 찾는 게 목적일 때 (부정 사용 탐지 등)

```
# IQR로 이상치 찾고 잘라내기
q1 = df['price'].quantile(0.25)
q3 = df['price'].quantile(0.75)
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
df['price'] = df['price'].clip(low, high)
```

이 단계에서 중복된 행 지우기(drop_duplicates), 숫자인데 글자로 저장된 열 바꾸기, "서울/서울시" 같은 표기 통일도 같이 해 두면 좋다.

4. 숫자 크기 맞추기와 글자를 숫자로 바꾸기

4.1 스케일링 (숫자 크기 맞추기)

나이는 20~60 정도인데 연봉은 2,000만~1억이다. 이대로 넣으면 모델은 숫자가 큰 연봉만 중요하다고 착각하기 쉽다. 그래서 얼마나 숫자 크기를 비슷하게 맞춰 주는데, 이것을 **스케일링**이라고 한다.

방법	계산	설명
Min-Max	$(x - \text{최솟값}) / (\text{최댓값} - \text{최솟값})$	모든 값을 0~1 사이로 바꾼다. 사진 픽셀(0~255)을 0~1로 바꿀 때 이걸 쓴다.
Standard	$(x - \text{평균}) / \text{표준편차}$	평균이 0, 표준편차가 1이 되게 바꾼다. 가장 많이 쓰는 방법이다.
Robust	$(x - \text{중앙값}) / \text{IQR}$	평균 대신 중앙값을 써서 이상치에 덜 흔들린다.
로그 변환	$\log(1 + x)$	큰 값을 많이 줄이고 작은 값을 조금 줄여서 한쪽으로 쏠린 모양을 퍼준다.

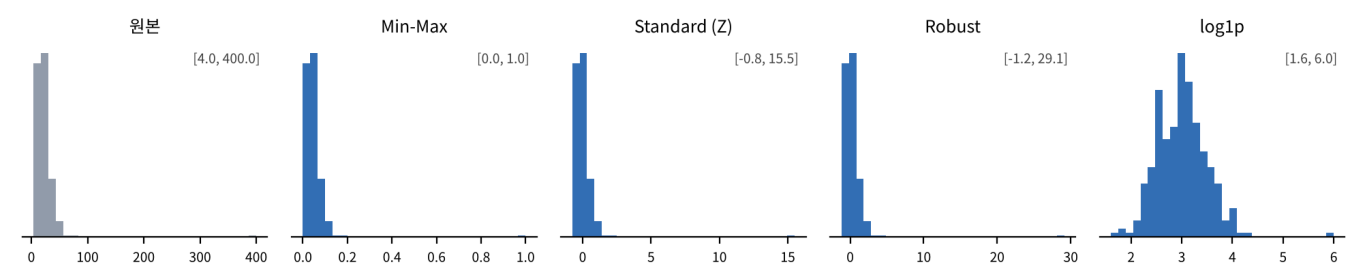


그림 2. 같은 데이터에 네 가지 방법을 써 본 결과. 앞의 세 방법은 숫자 범위만 바뀌고 모양은 그대로다. 로그 변환은 모양 자체가 바뀐다. 참고로 결정트리나 랜덤포레스트 같은 트리 모델은 "이 값보다 크냐 작냐"로만 판단해서 스케일링을 안 해도 된다.

4.2 인코딩 (글자를 숫자로)

"빨강, 초록, 파랑" 같은 글자 데이터를 숫자로 바꾸는 방법이다.

방법	예시	언제
레이블 인코딩	빨강=0, 초록=1, 파랑=2	간단하지만 "파랑(2)이 빨강(0)보다 크다" 같은 없는 순서가 생길 수 있다. 트리 모델이나 정답 데이터에 쓴다.
순서 인코딩	고졸=0, 학사=1, 석사=2	원래 순서가 있는 데이터일 때
원-핫 인코딩	빨강=[1,0,0] 초록=[0,1,0] 파랑=[0,0,1]	순서가 없는 데이터일 때. 종류가 너무 많으면 열이 엄청 늘어난다.

딥러닝 분류 모델의 정답도 이렇게 숫자로 되어 있다. 예를 들어 사진 분류에 많이 쓰는 ImageNet 데이터는 1,000개 종류에 0번부터 999번까지 번호가 붙어 있다.

4.3 새로운 열 만들기

있는 열을 조합해서 더 쓸모 있는 열을 만들 수도 있다. 생년월일로 나이를 계산하거나, 날짜에서 요일을 뽑거나, 키와 몸무게로 BMI를 만드는 식이다. 나이를 20대, 30대처럼 묶는 것도 자주 쓴다.

5. 데이터 나누기와 주의할 점

5.1 학습용, 검증용, 시험용

모델이 얼마나 잘하는지 알려면 **한 번도 본 적 없는 데이터**로 시험을 봐야 한다. 공부한 문제집으로 시험을 보면 점수가 높게 나오는 게 당연하기 때문이다. 그래서 데이터를 세 개로 나눈다.

이름	비율	비유	역할
학습용 (Train)	60~80%	교과서, 문제집	모델이 실제로 공부하는 데이터
검증용 (Validation)	10~20%	모의고사	설정을 바꿔 가며 어떤 게 나은지 비교
시험용 (Test)	10~20%	수능	맨 마지막에 딱 한 번 점수 확인

- 나눌 때 각 묶임에 정답 비율이 비슷하게 들어가도록 한다. (stratify=y 옵션)
- 날짜 순서가 있는 데이터는 섞지 말고 과거로 학습, 미래로 시험을 본다.
- 같은 사람 사진이 학습용과 시험용에 나눠 들어가면 점수가 실제보다 높게 나온다. 사람 단위로 나누는 게 맞다.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)
```

5.2 데이터 누수 (시험 문제 미리 보기)

모델이 학습할 때 시험용 데이터의 정보를 몰래 보게 되는 걸 **데이터 누수**라고 한다. 이러면 연습 때는 점수가 아주 좋은데 실제로 써 보면 성능이 뚝 떨어진다. 가장 흔한 실수는 순서 문제다.

틀린 순서

전체 데이터로 평균을 구해서 스케일링하고, 그다음에 학습용/시험용으로 나눈다.

맞는 순서

먼저 나누고, 학습용 데이터로만 평균을 구한 다음, 그 값으로 학습용과 시험용을 둘 다 바꾼다.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train) # 학습용으로 평균/표준편차 계산 + 변환
X_test = scaler.transform(X_test) # 시험용은 변환만 (계산 X)
```

scikit-learn의 Pipeline 을 쓰면 전처리와 모델을 한 번에 묶어서 이 순서를 자동으로 지켜 준다.

5.3 한쪽 데이터만 많을 때

카드 거래 100건 중 사기가 1건뿐이라면, 모델이 무조건 "정상"이라고만 해도 정확도가 99%가 나온다. 점수는 높는데 사기는 하나도 못 잡는 쓸모없는 모델이다. 이럴 때는

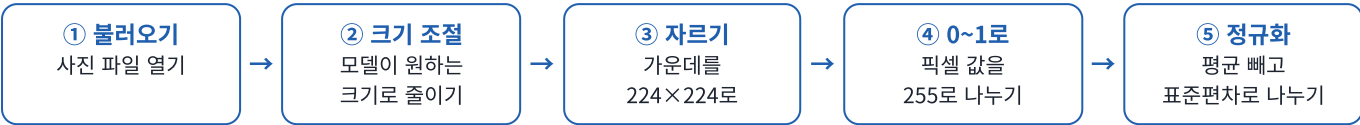
- 많은 쪽 데이터를 줄이거나, 적은 쪽 데이터를 늘린다. (비슷한 가짜 데이터를 만드는 SMOTE라는 방법도 있다.)
- 적은 쪽을 틀렸을 때 벌점을 더 크게 준다.
- 정확도 말고 다른 점수(재현율, F1 등)로 평가한다. 이건 6주차에 자세히 나온다.

데이터를 늘리거나 줄이는 건 반드시 나눈 다음 학습용에만 해야 한다.

6. 이미지 데이터 전처리

컴퓨터에게 사진은 숫자 덩어리다. 컬러 사진은 빨강(R), 초록(G), 파랑(B) 세 장의 판이 겹쳐 있고, 각 칸(픽셀)에는 0~255 사이 숫자가 들어 있다. 224×224 크기 컬러 사진 한 장이면 $224 \times 224 \times 3 = 150,528$ 개의 숫자다.

6.1 사진을 모델에 넣기까지



단계	설명
색 순서 확인	OpenCV로 사진을 열면 RGB가 아니라 BGR 순서로 읽힌다. 대부분의 모델은 RGB로 학습했기 때문에 꼭 바꿔 줘야 한다. 실수가 자주 나오는 부분이다. <code>img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)</code>
크기 맞추기	모델에 여러 장을 한 번에 넣으려면 크기가 다 같아야 한다. 그냥 늘리면 사진이 찌그러지니까 짧은 쪽을 맞추 다음 가운데를 잘라낸다.
0~1로 바꾸기	4장에서 본 Min-Max 스케일링이다. <code>transforms.ToTensor()</code> 를 쓰면 255로 나누는 것까지 알아서 해 준다.
정규화	이미 학습된 모델을 가져다 쓸 때는 그 모델이 학습할 때 쓴 평균과 표준편차로 똑같이 맞춰야 한다. ImageNet으로 학습된 모델은 아래 값을 쓴다. <code>mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]</code>

6.2 데이터 늘리기 (데이터 증강)

사진을 조금씩 바꿔서 학습 데이터를 늘리는 방법이다. 같은 고양이 사진이라도 좌우로 뒤집거나, 조금 돌리거나, 밝기를 바꾸면 모델 입장에서는 새로운 사진이 된다. 이렇게 하면 모델이 문제집 답만 외우는 걸(과적합) 막을 수 있고, 조명이나 각도가 달라져도 잘 알아 보게 된다.

종류	예시
모양 바꾸기	좌우 뒤집기, 회전, 확대/축소, 일부만 잘라내기
색 바꾸기	밝기, 대비, 채도 조절, 흑백으로 바꾸기
가리기·흐리게	사진 일부를 지우기, 흐리게 하기

```
from torchvision import transforms
train_tf = transforms.Compose([
    transforms.RandomResizedCrop(224),          # 랜덤으로 잘라서 224×224
    transforms.RandomHorizontalFlip(),          # 좌우 뒤집기
    transforms.ColorJitter(brightness=0.3),     # 밝기 바꾸기
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225]),
])
```

주의할 점이 몇 가지 있다. 증강은 학습용 데이터에만 하고 시험용에는 하지 않는다. 또 뜻이 바뀌는 변형은 안 된다. 숫자 6을 뒤집으면 9가 되기 때문이다.

7. 정리

단계	스스로 물어볼 것	쓰는 코드
살펴보기	열이 몇 개고 어떤 값들이 있지? 빈 칸이나 중복은?	<code>df.info()</code> , <code>df.describe()</code>
빈 값	왜 비었지? 지울까, 채울까?	<code>dropna()</code> , <code>fillna()</code>
이상치	잘못 들어간 값일까, 진짜 드문 값일까?	박스플롯, <code>clip()</code>
스케일링	열마다 숫자 크기가 너무 다르지 않나?	<code>StandardScaler</code> , <code>MinMaxScaler</code>
인코딩	글자로 된 열이 있나? 순서가 있는 값인가?	<code>OneHotEncoder</code>
나누기	시험용 데이터를 따로 빼 뒀나?	<code>train_test_split</code>
사진	RGB 순서 맞나? 크기와 정규화가 모델과 같나?	<code>Resize</code> , <code>ToTensor</code> , <code>Normalize</code>
증강	학습용에만 했나? 뜻이 바뀌지는 않나?	<code>RandomHorizontalFlip</code> 등

7.1 이것만은 기억하기

1. 데이터는 먼저 나누고, 평균 같은 값은 학습용으로만 계산한다.
2. 빈 값이나 이상한 값은 지우기 전에 왜 생겼는지부터 본다.
3. 학습할 때와 실제로 쓸 때 전처리 방법이 똑같아야 한다.
4. 사진 증강은 학습용 데이터에만 한다.
5. 남이 학습시킨 모델을 쓸 때는 그 모델이 쓴 전처리를 그대로 따른다.